

Design Rationale

REQ1: The Life in The Valley

A. Creature Production

Design 1: In-class reproduction logic Implement offspring spawning directly in SpiritGoat class.

Pros	Cons
Straightforward: no additional classes required.	Reproduction logic intermingles with movement, reducing clarity and testability.
	Modifications to reproduction triggers require changing SpiritGoat, breaking Open/Closed Principle

Design 2: Producible interface with ProduceBehaviour

Pros	Cons
Encapsulates reproduction in ProduceBehaviour, adhering to Single Responsibility.	Introduces interface and behavior classes, adding initial complexity.
Facilitates extension: new producers need no changes to SpiritGoat.	

B. Egg Implementation

Design 1: Implement hatching directly in specific egg classes (OmenSheepEgg, GoldenEgg).

Pros	Cons
Avoids the "fat interface" problem, i.e., inheriting too many unnecessary methods	Increased code duplication; shared functionalities must be re-implemented in each egg class
Avoids unnecessary constraints that inheritance may introduce, such as the hatching mechanism	Difficult to implement unified egg-related functionalities, such as a common hatching mechanism
	Difficult to manage all egg types uniformly, e.g., cannot use polymorphism effectively

Design 2: Abstract Egg class with internal ticking and hatching logic

Pros	Cons
Encapsulates lifecycle: aging, hatching, and placement logic co-located.	More complex lifecycle management.
Supports future egg variants via inheritance.	
Harmonizes with real-time tick progression.	

C. Egg Consumption Mechanism

Design 1: Generic ConsumableItem base to handle eating effects.

Pros	Cons
Reuses consumption logic for all edible items.	May overgeneralize effects, forcing items to fit a uniform model.
Keeps effect application consistent.	Unique item effects can complicate the base class.

Design 2: Eatable interface with EatAction and an EatAction class where each edible implements eat() to specify its effect.

Pros	Cons
Single Responsibility: consumption logic separated from item data.	Introduce more complex abstractions.
Items define their own eating effects, supporting diversity.	Slightly more setup.
Promotes reuse across different consumables.	

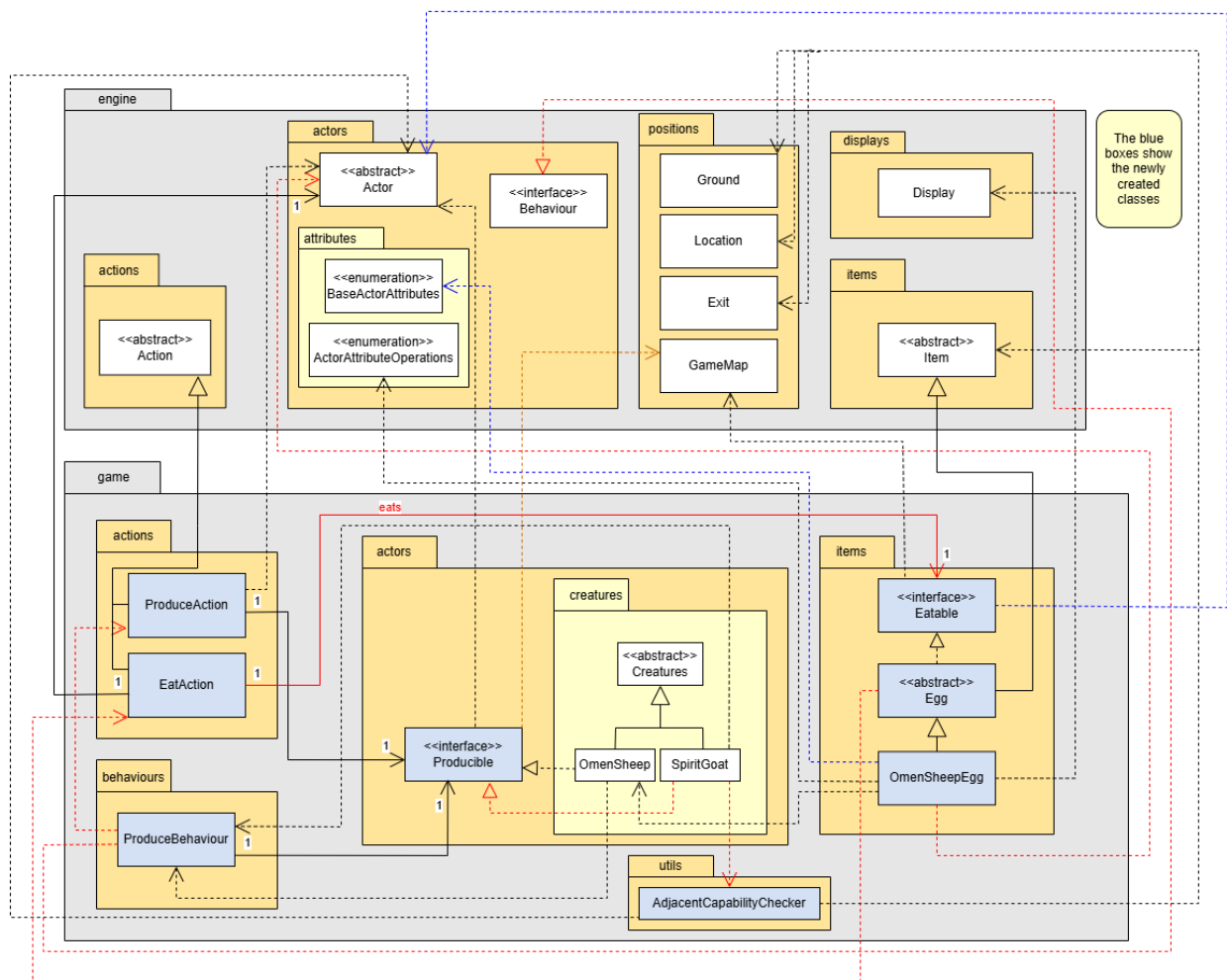
D. AdjacentCapabilityChecker Implementation

Design 1: Utility class A utility class that checks adjacency between entities without handling item effects.

Pros	Cons
No need to create instances, direct method calls	Tightly coupled with the caller
Centralized code, easy to locate	
Low memory usage (no need to create multiple instances)	

Design 2: Embed checking logic directly in methods Implement adjacency checking logic directly in methods that require it.

Pros	Cons
Simplifies immediate implementation	Violates DRY principle through code duplication
No additional classes to manage	Changes to checking logic require updates in multiple places
Direct access to local context	Reduced testability as logic is scattered



The above diagram shows the final design of Req1 in which we chose Design 2 for **Creature Production**, **Egg Implementation** and **Consumption Mechanism** whereas we use Design 1 for **AdjacentCapabilityChecker** implementation.

The architecture we've developed strictly adheres to SOLID principles, which provides significant benefits for maintainability and extensibility. For instance, the Single Responsibility Principle (SRP) is demonstrated through our clear separation of concerns: `ProduceBehaviour` exclusively manages and `EatAction` handles consumption effects.

This separation prevents any single class from becoming a "god class" with too many responsibilities, making the codebase easier to understand, test, and maintain.

Consider a hypothetical scenario where we need to modify how creatures reproduce. In a non-SRP design, we would need to modify the SpiritGoat class directly, potentially introducing bugs in unrelated functionality. Our implementation isolates reproduction logic in ProduceBehaviour, allowing targeted changes without disturbing movement or rendering code.

The Open/Closed Principle (OCP) is respected through our behavior injection approach with the Producible and Eatable interfaces. When a new creature type needs to be added, we can simply implement these interfaces without modifying existing framework code. For example, if requirements change to include a new "PhoenixBird" that reproduces differently, we can make it implement the Producible interface and implement its own produce method without changing a single line of code in the core framework.

Our design demonstrates the Liskov Substitution Principle (LSP) by ensuring subclass implementations can seamlessly replace their parent classes. OmenSheepEgg and GoldenBeetleEgg inherit from the abstract Egg class and can be used anywhere Egg is expected without altering program behavior. This consistency supports polymorphic behavior throughout the system, allowing methods that work with Egg to handle any specific egg subtype correctly.

The Interface Segregation Principle (ISP) is reflected in our carefully scoped interfaces. For example, Producible exposes only methods related to production capabilities, while Eatable focuses exclusively on consumption behaviours. This prevents the "fat interface" problem where implementers are forced to provide irrelevant functionality.

The Dependency Inversion Principle (DIP) is evident in how high-level modules like the game engine and action scheduler depend on abstractions (ProduceBehaviour, EatAction, Condition) rather than concrete implementations. This inversion allows the engine to remain agnostic about specific producers or consumables, enhancing system flexibility and testability. For instance, the game engine doesn't need to know whether a SpiritGoat or OmenSheep is producing offspring as it simply works with the Producible interface.

We've also prioritized the Don't Repeat Yourself (DRY) principle throughout our design. By centralizing adjacency checking in the AdjacentCapabilityChecker utility class rather than embedding this logic in various methods, we've eliminated potential duplication and ensured consistent behavior. Similarly, the abstract Egg class provides common lifecycle functionality for all egg types, reducing code repetition across OmenSheepEgg and GoldenBeetleEgg implementations.

From a connascence perspective, our design carefully manages various coupling types to enhance maintainability. It addresses Connascence of Name through consistent method naming conventions across interfaces and implementation classes. Connascence of Algorithm and Type is managed through interfaces that enforce specific method signatures and return types, ensuring uniform interaction patterns. Connascence of Timing and Value is handled through well-parameterized configurations, making behavior execution predictable.

If new requirements emerge, such as adding new creature types or changing reproduction mechanics, our design ensures minimal impact on existing systems. For example, if business requirements change to make reproduction dependent on time of day, we would only need to modify or extend the `canProduce()` method in the respective classes.

Our design comprehensively addresses all requirements while establishing a robust foundation that anticipates future changes. The architecture provides clear extension points for new features while maintaining a cohesive overall structure, balancing immediate implementation needs with long-term maintainability goals.